G^2-Mapping: General Gaussian Mapping for Monocular, RGB-D, and LiDAR-Inertial-Visual Systems

Lin Chen, Boni Hu, Jvboxi Wang, Shuhui Bu, *Member, IEEE*, Guangming Wang, *Graduate Student Member, IEEE*, Pengcheng Han, and Jian Chen

***Abstract*— In this paper, we introduce G^2-Mapping, a novel method to comprehensively support online monocular, RGB-D, and LiDAR-Inertial-Visual systems, employing 3D gaussian points as scene representation. There are several issues when applying 3d gaussian splatting (3DGS) techniques to simultaneous localization and mapping (SLAM) 1) for monocular, the lack of depth information makes scene initialization difficult and large baseline positioning challenging; 2) differentiable rendering with respect to depth and pose has not been implemented in 3DGS, making it difficult to directly apply to the SLAM system; 3) strategy for updating the scene with incoming online frames is not present, which may lead to memory overflow. In order to overcome problems mentioned above, we formulate a mathematical derivation and propose a differentiable rendering approach that leverages both depth and color to optimize the scene and pose. We introduce a simplified odometry that provides a metric depth estimation for monocular and enhance the low-overlap scene availability. A scale consistency and uncertainty weighted optimization is further proposed to eliminates the impact of inaccurate depth prediction. Our proposed scene updating strategy effectively prevents rapid memory growth. Tracking and mapping are performed alternatively to achieve precise localization and synchronous high-fidelity map reconstruction. Extensive experiments demonstrate that our G^2-Mapping surpasses feature-based SLAM in localization precision and exceeds state-of-the-art neural SLAM methods in the fidelity of view synthesis.**

***Note to Practitioners*—This paper is dedicated to tackling the efficiency challenges in multi-source SLAM and map reconstruction, aiming to generate pose and high-fidelity maps synchronously. We introduce G2-Mapping, a novel framework that leverages the power of 3D Gaussian points for scene representation, offering a universal solution for monocular, RGB-D, and LiDAR-Inertial-Visual systems. By developing a comprehensive differentiable renderer and presenting a strategy for dynamic scene updating, G2-Mapping significantly advances the state-of-the-art in localization precision and view synthesis fidelity. Although the approach is highly promising, it currently relies on the accuracy of depth prediction networks and requires further optimization for handling sparse LiDAR data. Future research will focus on enhancing these aspects, aiming to seamlessly integrate G2-Mapping into practical applications within robotics, autonomous vehicles, and augmented reality, where robust and efficient SLAM solutions are paramount.**

***Index Terms*— Gaussian representation, online view synthesis, differentiable rendering.**

Received 16 August 2024; revised 11 December 2024 and 23 January 2025; accepted 6 February 2025. Date of publication 13 February 2025; date of current version 16 April 2025. This article was recommended for publication by Associate Editor Z. Liu and Editor J. Yi upon evaluation of the reviewers' comments. This work was supported by the Post-Doctoral Fellowship Program of the China Postdoctoral Science Foundation (CPSF) under Grant GZB20240986. *(Corresponding authors: Shuhui Bu; Guangming Wang.)*

Lin Chen, Boni Hu, Jvboxi Wang, Shuhui Bu, and Pengcheng Han are with the National Key Laboratory of Aircraft Configuration Design, School of Aeronautics, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: npuchenlin@mail.nwpu.edu.cn; huboni@mail.nwpu.edu.cn; wangjv0182@163.com; bushuhui@nwpu.edu.cn; hanpc1125@nwpu.edu.cn).

Guangming Wang is with the Department of Engineering, University of Cambridge, CB2 1TN Cambridge, U.K. (e-mail: gw462@cam.ac.uk).

Jian Chen is with the School of Clinical Medicine, University of Cambridge, CB2 1TN Cambridge, U.K. (e-mail: jc2247@cam.ac.uk).

Digital Object Identifier 10.1109/TASE.2025.3541255

I. Introduction

REAL-TIME localization and synchronous high-fidelity map reconstruction have long been the goals of SLAM systems. In recent years, attention has shifted to Neural Radiance Field (NeRF), which embeds point coordinates into high-dimensional space through frequency embedding, enabling them to capture high-frequency details at the cost of a large number of multi-layer perceptrons (MLPs). To reduce the computational cost of MLPs, recent work embeds them into artificially designed spatial structures, such as octrees [1], [2], tri-planes [3], hash grids [4] and so on. NICE-SLAM [4] adopts a multi-resolution feature grid SLAM to address the limitations of large scene local updates. ESLAM [3] and Co-SLAM [5] combine multi-scale axis-aligned encoding of scene, but this does not fundamentally solve the problem of low computational efficiency.

3D gaussian splatting is an appealing scene alternative representation that has an astonishing rendering efficiency, and can even reach 800 fps [6] (1080P) by reducing the dimension of spherical harmonics (SH), regularization, etc. It expresses the scene through tiled splats with gaussian points containing mean, covariance, opacity, and color. Similar to point cloud, it is easy to add, delete, and update locally, making it an ideal SLAM scene representation.

Although gaussian splatting employs differentiable rendering for color using CUDA, the lack of differentiable gradients for depth and pose constrains its direct application to SLAM. Furthermore, for monocular, the lack of depth information

presents a significant challenge to scene initialization. At the same time, the non-convex nature in photometric loss introduces additional complexity to pose optimization, particularly over large baselines. Additionally, efficient scene update strategies are still needed to process online input frames without causing memory overflow, while also ensuring the quick convergence of the scene.

To address those issues, we propose G^2-Mapping, a general mapping method based on gaussian scene representation, distinguished as the first to facilitate online tracking and mapping of data from monocular, RGB-D, or LiDAR-Inertial-Visual (LIV) sensors. We first derive formulas for differentiable color and depth rendering, including the Jacobian with respect to the camera pose. Second, to address the challenge of monocular scene initialization, we propose a scale consistency and uncertainty weighted depth optimization module, which counters inherent prediction inaccuracies. Finally, in terms of mapping, we delve into the rendering pipeline and propose a refined strategy for adding and deleting gaussian points to prevent rapid memory growth. Additionally, we propose depth-based gaussian initialization to accelerate scene convergence. Through alternating optimization of the camera pose and the scene, we achieve precise localization. Our method sets a new benchmark, outperforming current state-of-the-art methods in both localization accuracy and view synthesis quality. In summary, our contributions can be summarized as follows:

- We develop a comprehensive differentiable renderer that handles both color and depth. It includes the Jacobian with respect to the camera pose, enhancing the rendering process. By incorporating extra constraints during back-propagation, our renderer facilitates more accurate depth estimation and improves online pose estimation.
- We introduce a simplified odometry approach that provides metric depth estimation for monocular, addressing the challenge of scene initialization. Furthermore, We eliminate the impact of inaccurate depth by leveraging consistent observation.
- We propose a gaussian scene updating strategy that meticulously integrates depth-based gaussian initialization, the addition of gaussian points tailored to the rendering pipeline, and the removal of transparency. This strategy enables rapid convergence from novel views while preventing substantial memory growth.
- Experiments on monocular, RGB-D, and LiDAR datasets demonstrate that our method surpasses traditional SLAM methods in localization and exceeds the view synthesis quality of the latest neural-based SLAM methods.

II. Related Works

A. Traditional Visual SLAM

Existing visual SLAM systems commonly rely on either discrete handcrafted feature extraction or deep learning embeddings, following the framework outlined in [7] regarding mapping and tracking. Generally, dense visual SLAM focuses more on establishing detailed and accurate 3D maps, with pose estimation not being as accurate as in sparse SLAM [8], [9]. There are two main approaches to 3D scene representation for visual SLAM, namely using voxel grids [10], [11] and point clouds [12], [13], [14]. Voxel-based geometric representations are often highly efficient but are computationally expensive and challenging to directly determine the resolution of voxel representation when the scene is unknown. In contrast, point-based representations can adaptively adjust resolution and scene size by dynamically allocating points, making them well-suited for use in online SLAM [13]. Many recent works have focused on integrating deep learning techniques into dense visual SLAM systems to achieve more accurate and robust mapping and tracking. For example, SceneCode [15] and DROID-SLAM [16] have made significant contributions in this field, achieving more accurate and robust mapping and tracking. However, optimizing representations to capture high-fidelity correlations between primitives remains challenging.

B. Neural Radiance Field Based SLAM

Traditional 3D reconstruction methods often involve the back-projection of 2D images into three-dimensional space and the weighted fusion of those projected results [17], [18]. This reconstruction approach often lacks the ability to express scene details. Neural Radiance Fields (NeRF) [19] has become a popular differentiable rendering method [20], dedicating to the generation of high-fidelity images from novel view. NeRF encodes color and opacity in the scene using MLPs. NeRF technology excels in expressing scene details and consistency. However, its training is time consuming, and it struggles to represent large scenes. Various approaches have addressed the issues of training speed [21], [22], [23] and large scene representation capacity [24], [25].

NeRF based SLAM has also achieved significant progress. iMAP [26] is the first method that incorporates NeRF technology into tracking and mapping. To improve scalability, NICE-SLAM [4] encodes maps with neural networks, avoiding the need for sparse feature matching and keypoint extraction required in traditional SLAM, thus achieving more robust and efficient mapping and localization. Recently, Point-SLAM [27] provids better 3D reconstruction by using neural point clouds and performing volume rendering with feature interpolation. However, the use of neural networks for implicit representation and rendering severely limits the improvement in runtime speed, making it difficult to meet real-time requirements.

C. 3D Gaussian Splatting

Recently, 3D gaussian splatting [28] has shown tremendous potential in high-quality real-time 3D scene synthesis. Based on differentiable rendering techniques, 3DGS achieving state-of-the-art visual quality and fast high-resolution rendering performance. Owning to the explicit scene representation and speed advantages of 3DGS, it has great potential for application in SLAM. In recent months, SplaTAM [29] has achieved real-time localization and mapping with RGB-D data by introducing silhouette masks and optimizing camera poses. However, it does not implement a backward pass for depth. Gaussian splatting SLAM [6] claims to support both monocular and RGB-D data, but when depth information is lacking, it is difficult to track under large motion with

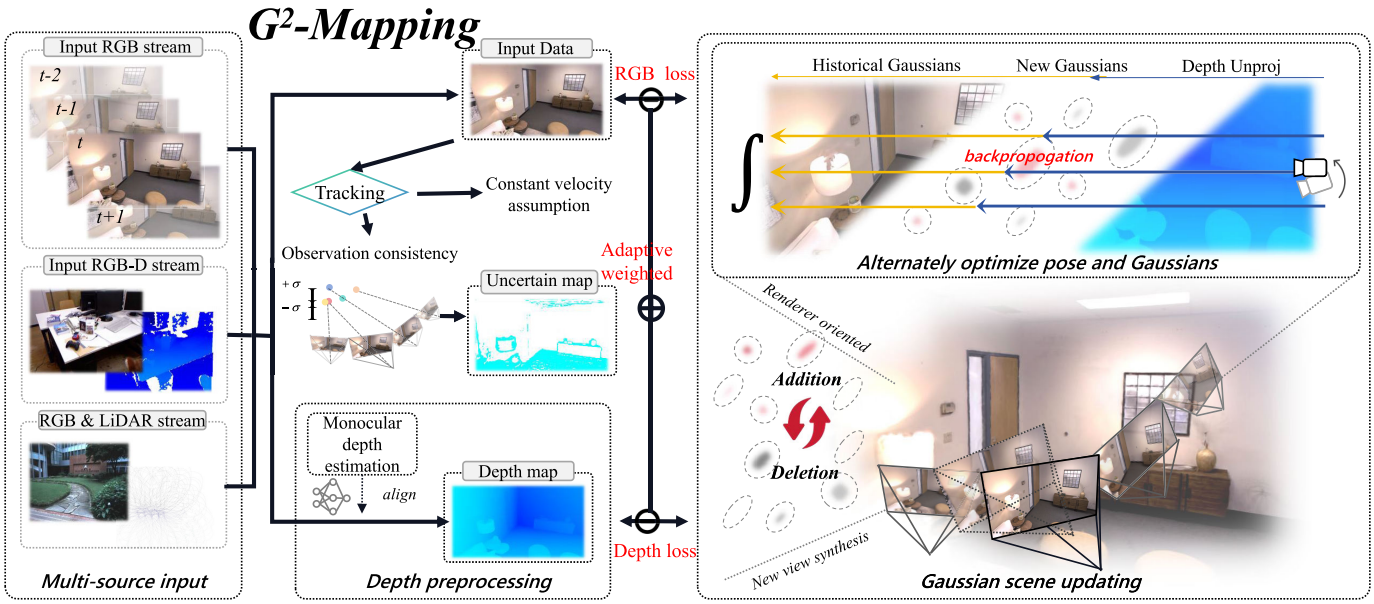

Fig. 1. **System overview.** Our method takes RGB/RGB-D/LIV data as input and outputs camera pose and scene representation. These input data first pass through a specific pre-tracking module to obtain the initial pose. If pre-tracking fails, the pose is initialized using the constant velocity assumption. For monocular systems, we also integrate a metric depth estimation module to provide initial depth information. We further improve the system performance by assigning pixel weights based on multi-view consistency. To achieve joint mapping and tracking, we render the predicted color and depth and alternately optimize the pose of the new frame and scene parameters.

photometric error. Moreover, the method inserts points at the average depth, and these noise points will cause the optimization to struggle. LIV-GaussMap [30] is the first LIV system based on 3DGS, but it does not make full use of depth, relying solely on color to optimize the scene, which affects the accuracy of localization and scene rendering.

In contrast, we propose a comprehensive differentiable renderer with Taichi [31], and fully utilize the backpropagation information from output depth, color, and intermediate variables such as pose. Moreover, it is the first general 3DGS mapping framework that supports multi-source data input.

III. Methodology

As shown in Fig. 1, we uses gaussian points (Section III-A) as the scene representation, and derive a comprehensive differentiable depth and color (Section III-B). After different types of data input, the odometry provides an initial pose and sparse depth. For monocular, we propose a scale-consistent and uncertainty-weighted depth optimization method (Section III-C), which reduces the error estimation of depth prediction. Map-based re-optimization (Section III-D) improves the localization accuracy from the odometry, and then the frame is added to the scene. Our proposed scene updating (Section III-D) strategy effectively prevents rapid memory growth and enables rapid convergence. Scene and pose optimization (Section III-E) are performed alternately, achieving precise localization and synchronous high-fidelity map reconstruction.

A. Gaussian Scene Representation

The advantages of gaussian scene representation include rapid rendering and convergence, as well as ease of modification. The entire scene is represented by $\mathcal{G}$, where each G_i denotes the i-th gaussian point within the scene. Each gaussian point comprises four components: the position $\mathbf{P}_i \in \mathbb{R}^3$, the covariance matrix $\Sigma_i \in \mathbb{R}^{3\times3}$, the opacity α_i, and the color $\mathbf{c}_i$.

The set of cameras present in the scene is denoted by $\mathcal{K}$. Each camera frame $\mathcal{K}_j$ is defined by an intrinsic matrix $\mathbf{K}_j \in \mathbb{R}^{3\times3}$, which remains constant and known when the scene is captured by the same camera, and a transformation matrix $\mathbf{T}_j \in SE(3)$ that specifies the camera's pose relative to the world coordinate system.

$$\begin{aligned}\mathcal{G} &= \{\mathcal{G}_i : (\mathbf{P}_i, \Sigma_i, \alpha_i, \mathbf{c}_i) | i = 1, \ldots, n\},\\ \mathcal{K} &= \{\mathcal{K}_j : (\mathbf{K}, \mathbf{T_j}) | j = 1, \ldots, m\},\end{aligned} \tag{1}$$

during rendering, the 2D mean μ' is derived from the pose $\mathbf{T}_{cw}$ and the projection π. The 2D covariance Σ' follows a linear approximation [32] using Jacobian $\mathbf{J}$. Opacity at u_i is a. The overall projection process is shown in Eq. (2).

$$\mu' = \pi(\mathbf{T}_{cw}\mathbf{P}),\ \ \Sigma' = \mathbf{J}\mathbf{R}_{cw}\Sigma\mathbf{R}_{cw}^T\mathbf{J}^T,\ \ a_i = f_{exp}(\mathbf{u_i}|\mu_i', \Sigma_i')\alpha_i. \tag{2}$$

Given the mean μ' and covariance Σ' on the 2D plane, the rendering process sequentially accumulates the color c_i, depth d_i, and opacity a_i for each gaussian point indexed by i along the ray, based on their respective depth values. This results in the rendered color $\hat{C}$, depth $\hat{D}$, and opcacity $\hat{T}$. The rendering equations are shown in Eq. (3).

$$\begin{aligned} w_i &= \prod_{j=1}^{i-1}(1-a_j), \quad \hat{T} = \sum_{i=1}^{n} a_i w_i,\\ \hat{C} &= \sum_{i=1}^{n} c_i a_i w_i, \quad \hat{D} = \sum_{i=1}^{n} d_i a_i w_i, \end{aligned} \tag{3}$$

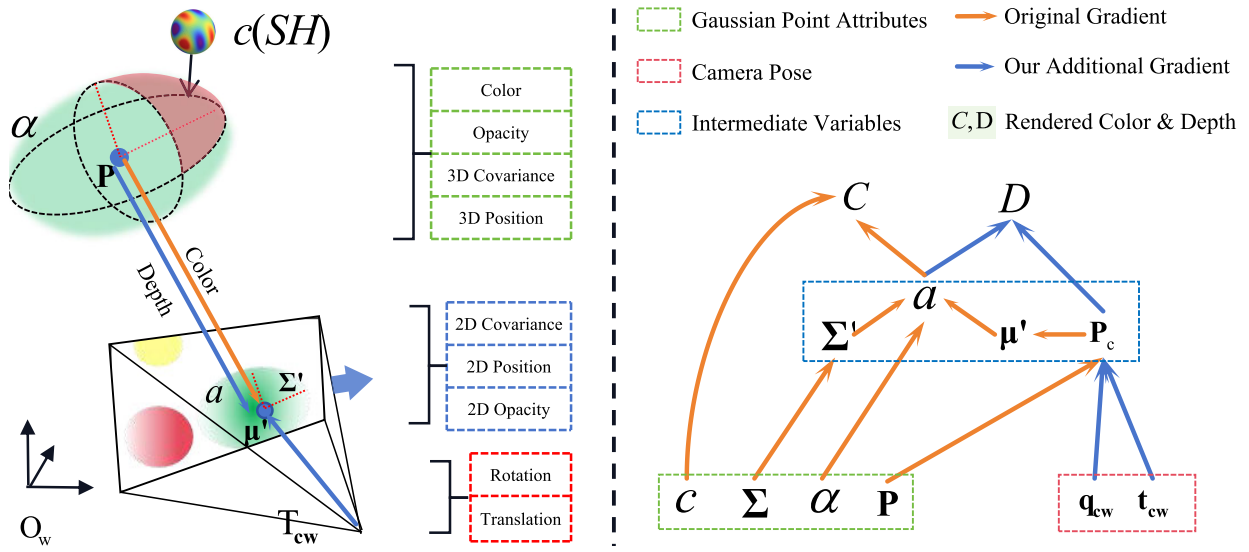

Fig. 2. **Diagram of the chain rule for color and depth.** The paper [28] implements the orange part, and we add the blue part. The optimization of depth and pose allows us to incrementally add new frames while ensuring rapid convergence of the scene.

where w_i represents the accumulated opacity. Leveraging these rendering equations, we extend the differentiable color rendering found in orignal 3DGS [28] to include differentiable rendering for depth and opacity. By incorporating depth constraints, we achieve stable pose estimation while ensuring the fidelity of the synthesized novel views.

B. Differentiable Depth and Pose Optimization Analysis

PyTorch lacks an automatic differentiation mechanism for gaussian scene. The original 3DGS [28] derived and implemented the derivatives of color with respect to gaussian point attributes ($\mathbf{P}$, Σ, α, c) using CUDA. We not only introduce differentiable depth to optimize gaussian point attributes but also derive an optimization method for camera pose parameters, achieving accurate depth rendering and online pose estimation.

Figure 2 illustrates the overall chain rule process, where $\mathbf{P_c}$ represents the mean of gaussian points in the camera coordinate system. The camera transformation matrix $\mathbf{T_{cw}}$ is expressed using a quaternion $\mathbf{q_{cw}} \in \mathbb{R}^4$ and a translation matrix $\mathbf{t_{cw}} \in \mathbb{R}^3$.

1) Differentiable Depth: We introduce dense depth information to constrain the scene for RGB-D and monocular. Depth information ensures the geometric consistency of the scene and provides improved guidance for camera pose estimation.

Initially, we examine the derivative of depth D in relation to the attributes of the gaussian points. This process involves the transformation of the gaussian point coordinates into the camera's coordinate frame, followed by their subsequent projection onto the two-dimensional plane as delineated by Eqs. (2) and (3). Through this, the derivative of depth D concerning the gaussian point attributes is explicated in Eq. (4).

$$\begin{aligned}\frac{\partial D}{\partial \mathbf{P}} &= \frac{\partial D}{\partial a}\frac{\partial a}{\partial \mathbf{P}} + \frac{\partial D}{\partial d}\frac{\partial d}{\partial \mathbf{P}},\\ \frac{\partial D}{\partial \Sigma} &= \frac{\partial D}{\partial a}\frac{\partial a}{\partial \Sigma},\\ \frac{\partial D}{\partial \alpha} &= \frac{\partial D}{\partial a}\frac{\partial a}{\partial \alpha}.\end{aligned} \tag{4}$$

Following the chain rule as illustrated in Fig. 2, we calculate the derivative of depth D with respect to the gaussian point attributes using Eqs. (2) and (3).

$$\begin{aligned}\frac{\partial D}{\partial a_i} &= d_i w_i - \frac{\sum_{j=i+1}^{n} d_j a_j w_i}{1-a_i},\\ \frac{\partial D}{\partial d_i} &= a_i w_i,\\ \frac{\partial d_i}{\partial \mathbf{P}_i} &= \mathbf{R}_{cw}[:,3].\end{aligned} \tag{5}$$

By iteratively applying these adjustments through gradient descent, the scene's parameters can be fine-tuned to minimize the difference between the rendered and expected depth, improving the accuracy and realism of the rendered scene.

2) Camera Pose Gradient: For convenience, we differentiate rotation quaternion $\mathbf{q_{cw}}$ and translation $\mathbf{t_{cw}}$ separately rather than the transformation matrix $\mathbf{T_{cw}}$.

Let $\mathbf{q_{cw}} = [w, \mathbf{v}]$, and $\mathbf{q_{cw}}^*$ is the conjugate of the quaternion. $[\cdot]_\times$ denotes the antisymmetric matrix. Then we have:

$$\begin{aligned}\frac{\partial \mathbf{P_c}}{\partial \mathbf{q_{cw}}} &= \frac{\partial \mathbf{q_{cw}} \otimes \mathbf{P} \otimes \mathbf{q_{cw}}^*}{\partial \mathbf{q}}\\ &= 2[\mathbf{J}_{imag}|\mathbf{J}_{real}] \in \mathbb{R}^{3\times 4},\end{aligned} \tag{6}$$

$$\frac{\partial \mathbf{P_c}}{\partial \mathbf{t_{cw}}} = \mathbf{I}_3 \in \mathbb{R}^{3\times 3}, \tag{7}$$

where $\mathbf{J}_{imag} = \mathbf{v}^\top \mathbf{P}\mathbf{I}_3 + \mathbf{v}\mathbf{P}^\top - \mathbf{P}\mathbf{v}^\top - w[\mathbf{P}]_\times$, $\mathbf{J}_{real} = w\mathbf{P} + \mathbf{v}\times\mathbf{P}$.

We fully derive the gradients in the rendering process, enabling us to optimize the scene's parameters through gradient descent. In practice, we found that depth continuously reduces the opacity α of observed gaussian points, leading to incorrect deletions in the deletion step. Therefore, in our implementation, we empirically multiply the value of $\frac{\partial D}{\partial \alpha_i}$ by a coefficient of 1e-4 to mitigate the impact of depth on opacity.

C. Monocular / RGB-D / LIV Preprocess

1) Pre-Tracking: The paper [33] proves that large pose mismatch will lead to failure pose registration for complex

image signals. Existing end-to-end SLAM [6], [29], [34] often assumes constant velocity for initial pose, which is difficult to handle large baseline data. In our system, we implement a custom, simplified odometry to estimate the frame to frame relative pose. For monocular, a simply feature based pre-tracking process is designed to maintain a local map to ensure scale consistency in tracking. For LiDAR and RGB-D, we employ the Iterative Closest Point (ICP) algorithm to estimate the relative pose between scans.

This process primarily serves three functions. (a) It addresses the scale consistency problem in depth estimation, ensuring the dense addition of gaussians in monocular mode; (b) for LiDAR-Inertial-Visual data, it allows for quick preprocessing by employing IMU-based undistortion methods; (c) it provides a more accurate subsequent pose estimate compared to the constant velocity assumption, avoiding multi-layer downsampling. Even if it fails, the algorithm can still degrade to assuming constant velocity to continue optimizing the pose through photometric and geometric constraints.

2) Monocular Initialization: On mono case, the absence of depth information hinders the initialization of Gaussian points by 3DGS. However, our pre-tracking odometry process allows us to overcome this issue by integrating a monocular depth estimation algorithm. The sparse points maintained by the odometry ensure scale consistency across adjacent frames, allowing us to align the results of depth estimation to a consistent scale. If pre-tracking fails, the scale consistency is calculated using the median of the scene depth. To ensure scale drift is minimized during long-term tracking, we continuously update the pre-tracking poses with the pose optimization results from the 3DGS frame-to-map tracking.

3) Uncertainty Assignment: Even with sparse point alignment, there are still many erroneous points. To address this, depth uncertainty is proposed to be assigned to each pixel. During optimization, pixels with greater uncertainty are assigned less weight, and vice versa, thereby reducing the impact of erroneous points on optimization. For LiDAR data, the uncertainty is set to a fixed value, as the depth values of LiDAR data are usually accurate.

When a posed frame from odometry is received, we project the depth maps of nearby frames onto the current frame and calculate the depth discrepancies between the current frame and the reference frames.

$$\sigma(\mathbf{p}) = \left[\mathbf{T}_{rc} \cdot \pi^{-1}(\mathbf{p}_c, d_c)\right]_z - d_r. \tag{8}$$

The pose of the current frame relative to the reference frame is denoted by $\mathbf{T_{rc}}$. The function π^{-1} reverses the projection process, while d_r and d_c represent the depths in the reference and current frames, respectively. Through this, we determine the depth uncertainty $\sigma(\mathbf{p})$ at a pixel $\mathbf{p}$.

D. Gaussian Scene Updating With Depth Prior

1) Map-Based Tracking: Fine-tune the pose obtained from the pre-tracking process will improve the accuracy. Once the pre-tracked pose is acquired, it is continuously refined by minimizing the discrepancy between color and depth. Typically, low opacity areas indicate incomplete initialization. We propose an opacity mask, refining only the regions with opacity greater than a_{thres}, to avoid the impact of uninitialized areas on the pose adjustment process. After the adjustment is complete, this updated pose is then used to enhance the pre-tracking process.

2) Gaussian Addition: SplaTAM and GS-SLAM [29], [34] directly add pixels with large differences between rendered and actual depth. This easily leads to non-convergence and memory overload when the depth values are inaccurate. We delve into the rendering pipeline and propose an efficient search for the gaussian scene $\mathcal{G}$, directly determining whether there is a gaussian point near the depth $D(\mathbf{p})$ during alpha-blending. If there is not, the pixel $\mathbf{p}$ will be unprojected and added to the scene. Otherwise, it will not be added to the scene. Additionally, we render an opacity map $\hat{T}$ based on the pose. Pixels $\mathbf{p}$ with opacity less than a_{thres} and uncertainty less than σ_{thres} are also added. Experiments on TUM-RGBD [35] and Replica [36] datasets show that this can reduce the number of gaussian points by 70 $\sim$ 95% compared to SplaTAM [29], while ensuring the fidelity of the rendering.

$$D(\mathbf{p}) \notin \{\mathcal{G}\} \quad or \quad \hat{T}(\mathbf{p}) < a_{thres} \quad or \quad \sigma(\mathbf{p}) < \sigma_{thres}. \tag{9}$$

3) Depth-Based Gaussian Initialization: To accelerate the convergence of novel view, we initialize the gaussian point by leveraging dense depth data, constructing a kd-tree from the newly added 3D points, and applying SVD on neighbors to get eigenvalues λ and eigenvectors ξ. These are then used to initialize scale s and orientation $\mathbf{R}$. The point's color initializes the SH coefficients, and we set α to 0.5.

4) Gaussian Deletion: GS-SLAM [34] manually reduces the opacity of points based on the distance from the depth. In contrast, we directly use Eq. (5). Erroneous points that fail depth or color criteria will continuously diminish in opacity as observations proceed, which is more elegant. We delete points with opacity less than 0.1 every 200 frames. Through experiments, we find that depth constraints are highly effective in removing erroneous points.

E. Optimization

Our proposed depth uncertainty $\sigma(\mathbf{p})$ enables us to control the weight of the loss function on a per-pixel basis. The Gaussian Negative Log Likelihood Loss (GNLL) possesses favorable properties, assigning greater weight where uncertainty is low, and vice versa. Hence, in conjunction with the depth uncertainty we propose, the depth and color loss at pixel $\mathbf{p}$ is shown in Eqs. (10) and (11).

$$\mathcal{L}_{\text{depth}}(\mathbf{p}) = \log\left(\sigma(\mathbf{p})^2\right) + \frac{\left(\hat{D}(\mathbf{p}) - D(\mathbf{p})\right)^2}{\sigma(\mathbf{p})^2}, \tag{10}$$

$$\mathcal{L}_{\text{color}}(\mathbf{p}) = \left\|\hat{C}(\mathbf{p}) - C(\mathbf{p})\right\|_1, \tag{11}$$

$\hat{C}$ and C represent the rendered color and the ground truth color, respectively, while $\hat{D}$ and D represent the rendered depth and the ground truth depth, respectively. $\mathcal{L}_{depth}$, $\mathcal{L}_{color}$ are the loss corresponding to depth and color. Thus, the total loss for a pixel can be expressed as Eq. (12).

$$\mathcal{L}(\mathbf{p}) = (1 - \lambda_s)\mathcal{L}_{color}(\mathbf{p}) + \lambda_s\mathcal{L}_{ssim}(\mathbf{p}) + \lambda_d\mathcal{L}_{depth}(\mathbf{p}). \tag{12}$$

TABLE I
CAMERA TRACKING RESULT ON TUM-RGBD [35]. OUR METHOD ACHIEVES STATE-OF-THE-ART RESULTS IN BOTH MONOCULAR AND RGB-D CASES. PARTICULARLY, OUR METHOD ALSO OUTPERFORMS SYSTEMS THAT USE DEPTH PRIORS IN THE MONOCULAR CASE. BEST RESULTS ARE HIGHLIGHTED AS **FIRST**

Input	Method	Publication	Avg. ↓		fr1/desk	fr1/desk2	fr2/xyz	fr3/off.
RGB-D	Kintinuous [37]	IJRR 2014	4.84		3.70	7.10	2.90	3.00
	ElasticFusion [38]	IJRR 2016	6.91		2.53	6.83	1.17	2.52
	ORB-SLAM3-VO [39]	TRO 2021	1.24		1.51	2.17	0.36	0.97
	NICE-SLAM [4]	CVPR 2022	11.21		4.26	4.99	31.73	3.87
	Vox-Fusion [1]	ISMAR 2022	12.08		3.52	6.00	1.49	26.01
	Point-SLAM [27]	ICCV 2023	3.42		4.34	4.54	1.31	3.48
	SplaTAM [29]	CVPR 2024	4.07		3.35	6.54	1.24	5.16
	Ours	-	1.21	↑ **2.42%**	1.45	2.17	0.32	0.90
Monocular	DROID-VO [16]	NIPS 2021	7.73		5.20	N/A	10.7	7.30
	DSO [40]	ICCV 2017	11.0		22.4	N/A	1.1	9.50
	GS-SLAM [34]	CVPR 2024	3.7		3.3	N/A	1.3	6.6
	Ours	-	1.87	↑ **49.46%**	2.21	N/A	0.95	2.44

The total loss $\mathcal{L}(\mathbf{p})$ for a pixel $\mathbf{p}$ combines color loss $\mathcal{L}_{color}(\mathbf{p})$, structural similarity loss $\mathcal{L}_{ssim}(\mathbf{p})$, and depth loss $\mathcal{L}_{depth}(\mathbf{p})$. The weights λ_s and λ_d adjust the importance of structural similarity and depth accuracy, respectively.

We only taken valid pixels for account, as shown in Eq. (13). The symbol $\mathbb{I}$ represents an indicator function, which equals 1 when the condition in the parentheses is true, and 0 when it is false.

$$\min_{\mathcal{G},\mathcal{K}} \mathcal{L} = \sum_{\mathbf{p}} \mathbb{I}(a > a_{thres}) \mathcal{L}(\mathbf{p}). \tag{13}$$

Alternating optimization of the gaussian scene $\mathcal{G}$ and camera extrinsic $\mathcal{K}$ is employed to accounts for their mutual influence. Scene optimization is performed on randomly selected frames that are in proximity to each newly added frame.

IV. EXPERIMENT

A. Experimental Setup

1) Datasets and Evaluation Metrics: We evaluate our method on three datasets: Replica [36], TUM-RGBD [35], and R3Live-Dataset [41]. We evaluate the rendering quality and localization accuracy.

The average absolute trajectory error (ATE, RMSE in [cm]) is utilized to evaluate the pose accuracy. The stability of localization is evaluated by reducing the frame overlap through selective frame decimation. The PSNR, SSIM and LPIPS [42] are employed to assess rendering performance. The number of gaussian points and runtime are used to evaluate the efficiency of reconstruction.

2) Implementation Details: Our differentiable renderer is implemented in the Taichi language [31], and the optimization is performed using the Adam optimizer with a learning rate of 1e-5. Our method runs on a desktop computer with an NVIDIA RTX 4090. We set α_{thres} to 0.1 and σ_{thres} is set according to the depth range of the scene. For the TUM-RGBD [35] and Replica [36] datasets, we set it to 0.3.

B. Evaluation of Localization

The TUM-RGBD dataset [35] is a challenging dataset due to the motion blur, low-textured RGB images, inferior depth sensor quality and extensive blank areas. Results in Table I demonstrate superior results when benchmarked against state-of-the-art SLAM techniques. In RGB-D case, our approach outperformed the Point-SLAM [27] and SplaTAM [29] methods, achieving a remarkable 64.6% reduction in trajectory error, decreasing from 3.42 cm to just 1.21 cm. Furthermore, when compared to the feature-based ORB-SLAM2 [43] method, our method achieved a substantial 38.9% decrease in trajectory error, from 1.98 cm to 1.21 cm.

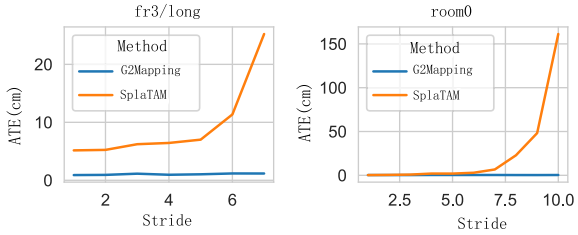

Fig. 3. **Stride and Accuracy.** As the stride increases, the accuracy of SplaTAM decreases, while our method remains stable.

In the monocular case, we tested the DOIRD-SLAM (w/o loop-closure) [16], DSO [40], and GS-SLAM [34]. Our method also achieved the best results in the monocular case. Compared to the latest monocular gaussian-based SLAM, our method achieved a 30.27% reduction in trajectory error, decreasing from 3.7 cm to 2.58cm. Due to the proposed opacity mask and differentiable depth, our method is able to further optimize the pose based on the reconstructed scene under the initial pose provided by the odometry, achieving superior results compared to other methods.

To evaluate the robustness of our algorithm against images with low overlap, we incrementally increased the stride between images during our experiments. Figure 3 demonstrates that SplaTAM [29] struggles with tracking at low overlap, leading to a significant increase in ATE error, while our algorithm ensures stable convergence.

C. Runtime Analysis

We compared the runtime performance of our method with that of SplaTAM [29] as shown in Table II. Our method outperformed SplaTAM on all three datasets, achieving an average improvement of 66.17%. The efficient performance comes from the stability of our method when adding new points and the initial value provided by the odometry, which requires less time to optimize the pose. Additionally, we provide a version

TABLE II

TIME CONSUMING PER FRAME. IT IS OBSERVABLE THAT OUR METHOD OUTPERFORMS SplaTAM [29] IN TERMS OF SPEED. ADDITIONALLY, WE PROVIDES AN ACCELERATED VERSION WITH A REDUCED SH DIMENSION, WHICH REDUCES THE TRAINING TIME BY HALF, ALBEIT WITH A MINOR SACRIFICE IN RENDERING QUALITY (∼1.2 DECREASE IN PSNR ON TUM)

Method	Publication	fr1/desk↓		fr1/desk2↓		fr2/xyz↓		Average↓	
SplaTAM	CVPR 2024	2.83s		3.15s		12.19s		6.06s	
Ours-(SH:16)		**2.13s**	↑ **24.73%**	**2.35s**	↑ **24.92%**	**1.68s**	↑ **86.22%**	2.05s	↑ **66.17%**
Ours-(SH:4)		**1.27s**	↑ **55.12%**	**1.40s**	↑ **55.56%**	**1.05s**	↑ **91.39%**	1.24s	↑ **79.54%**
Ours-(SH:1)		**1.04s**	↑ **63.25%**	**1.10s**	↑ **65.08%**	**0.83s**	↑ **93.19%**	0.99s	↑ **83.66%**

TABLE III

RENDERING PERFORMANCE ON REPLICA [36] AND TUM-RGBD [35]. WE EVALUATE NICE-SLAM [4], ESLAM [44], POINT-SLAM [27], AND SplaTAM [29] ON THE RGB-D DATA. TO ENSURE CONSISTENCY WITH NICER-SLAM [45], WE USE NOVEL VIEW TO EVALUATE THE MONOCULAR CASE. BEST RESULTS ARE HIGHLIGHTED AS **FIRST**, **SECOND** AND **THIRD**

Input	Method	Metrics	Replica [36] Avg.	*R0*	*R1*	*R2*	*Of0*	*Of1*	*Of2*	*Of3*	Of4	TUM RGB-D [35] Avg.	*fr1/ desk*	*fr1/ desk2*	*fr2/ xyz*	*fr3/ off.*
RGB-D	NICE-SLAM [4]	PSNR[dB]↑	24.41	22.39	22.36	23.92	27.79	29.83	20.33	23.47	25.21	13.59	13.83	12.00	17.87	12.89
		SSIM↑	0.80	0.68	0.75	0.80	0.86	0.88	0.79	0.80	0.85	0.55	0.57	0.51	0.72	0.55
		LPIPS↓	0.24	0.30	0.27	0.23	0.24	0.18	0.24	0.21	0.20	0.49	0.48	0.52	0.34	0.50
	ESLAM [44]	PSNR[dB]↑	27.80	25.25	25.31	28.09	30.33	27.04	27.99	29.27	29.15	13.42	11.29	12.30	17.46	17.02
		SSIM↑	0.93	0.88	0.90	0.93	0.96	0.92	0.94	0.95	0.95	0.60	0.67	0.63	0.31	0.46
		LPIPS↓	0.25	0.32	0.30	0.25	0.21	0.25	0.24	0.19	0.21	0.46	0.36	0.42	0.70	0.65
	Point-SLAM [27]	PSNR[dB]↑	35.17	32.40	34.08	35.50	38.26	39.16	33.99	33.48	33.49	15.63	13.87	14.12	17.56	18.43
		SSIM↑	0.98	0.97	0.98	0.98	0.98	0.99	0.96	0.96	0.98	0.67	0.63	0.59	0.710	0.75
		LPIPS↓	0.12	0.11	0.12	0.11	0.10	0.12	0.16	0.13	0.14	0.54	0.54	0.57	0.59	0.45
	SplaTAM [29]	PSNR[dB]↑	34.11	32.86	33.89	35.25	38.26	39.17	31.97	29.70	31.81	22.23	21.73	20.90	24.46	21.83
		SSIM↑	0.97	0.98	0.97	0.98	0.98	0.98	0.97	0.95	0.95	0.866	0.85	0.79	0.95	0.87
		LPIPS↓	0.10	0.07	0.10	0.08	0.09	0.09	0.10	0.12	0.15	0.205	0.25	0.26	0.10	0.21
	Ours RGB-D	PSNR[dB]↑	39.30	36.73	38.16	39.63	44.51	43.32	36.42	37.14	38.51	26.09	25.76	24.48	27.8	26.33
		SSIM↑	0.96	0.92	0.93	0.98	0.99	0.99	0.98	0.94	0.93	0.848	0.83	0.83	0.87	0.86
		LPIPS↓	0.02	0.02	0.01	0.01	0.01	0.01	0.02	0.02	0.02	0.095	0.10	0.11	0.07	0.10
Mono	NICER [45] NovelView*	PSNR[dB]↑	23.93	25.64	23.69	22.62	25.88	22.56	21.46	24.42	25.15	-	-	-	-	-
		SSIM↑	0.85	0.81	0.82	0.87	0.89	0.83	0.86	0.89	0.89	-	-	-	-	-
		LPIPS↓	0.20	0.25	0.23	0.20	0.19	0.16	0.20	0.18	0.19	-	-	-	-	-
	Ours NovelView*	PSNR[dB]↑	27.58	26.21	27.58	27.73	31.91	28.19	25.88	28.36	24.76	-	-	-	-	-
		SSIM↑	0.86	0.82	0.80	0.90	0.92	0.87	0.89	0.87	0.80	-	-	-	-	-
		LPIPS↓	0.05	0.05	0.04	0.06	0.03	0.04	0.05	0.05	0.07	-	-	-	-	-

TABLE IV

MEMORY COMPARISON ON REPLICA [36] AND TUM-RGBD [35]. WE REPRESENT MEMORY CONSUMPTION BY THE NUMBER OF POINTS IN THE SCENE [IN MILLIONS]. OUR METHOD ACHIEVES BETTER RESULTS WITH FEWER POINTS

Method	Replica [36] room0		Replica [36] office0		TUM RGB-D [35] fr1/desk		TUM RGB-D [35] fr1/desk2		TUM RGB-D [35] fr2/xyz	
SplaTAM [29]	5.12		6.22		0.97		1.39		6.37	
Ours	**1.45**	↑ **71.7%**	**1.39**	↑ **77.7%**	**0.33**	↑ **66.0%**	**0.35**	↑ **74.8%**	**0.24**	↑ **96.2%**

that reduces the dimensions of SH to speed up the algorithm's speed. While this entails a certain level of compromise in rendering accuracy, it has enabled rapid map construction.

D. Rendering Evaluation

As shown in Table III, for the RGB-D scenario, we compare our method with SplaTAM [29], ESLAM [44], Point-SLAM [27], and NICE-SLAM [4]. It is important to note that Point-SLAM utilizes depth ground truth for sampling, which leads to an unfair comparison. Despite this, our method still surpasses the second-best method by 5.19dB (↑15.2%) and 0.08 (↑80%) in PSNR and LPIPS, but slightly decrease in SSIM (↓2%).

In the monocular case, to maintain consistency with prior work [45], we present the novel view synthesis performance. Our approach comprehensively outperforms NICER-SLAM [45] by 3.65 (↑15.3%) in PSNR, 0.01 (↑1.1%) in SSIM, and 0.02 (↑75%) in LPIPS in novel view synthesis. This demonstrates that our method can achieve the best results in the monocular case, with the help of depth prediction priors.

As shown in Fig. 4, the latest gaussian-based SLAM method, SplaTAM [29], still has abnormal color points, blurry edges, and holes at the edges in the rendered results. Our method produces more realistic rendering results.

This is because our scene update method only adds necessary points, thereby reducing the training load. On the other hand, SplaTAM [29] employs a more extensive point-adding strategy, which leads to an excessive training burden and results in renderings that are not as good as those produced by our method.

We test the performance of our method against the original 3DGS [28] method and two latest lidar-based gaussian splatting methods LVI-GS [46] and Gaussian-LIC [47] on three R3Live-Dataset [41] sequences: *deg_00*, *deg_01* and *campus_00*. Considering the sparsity of LiDAR data, we adjust our strategy for densifying points more aggressively. In the tests with the original 3DGS method,

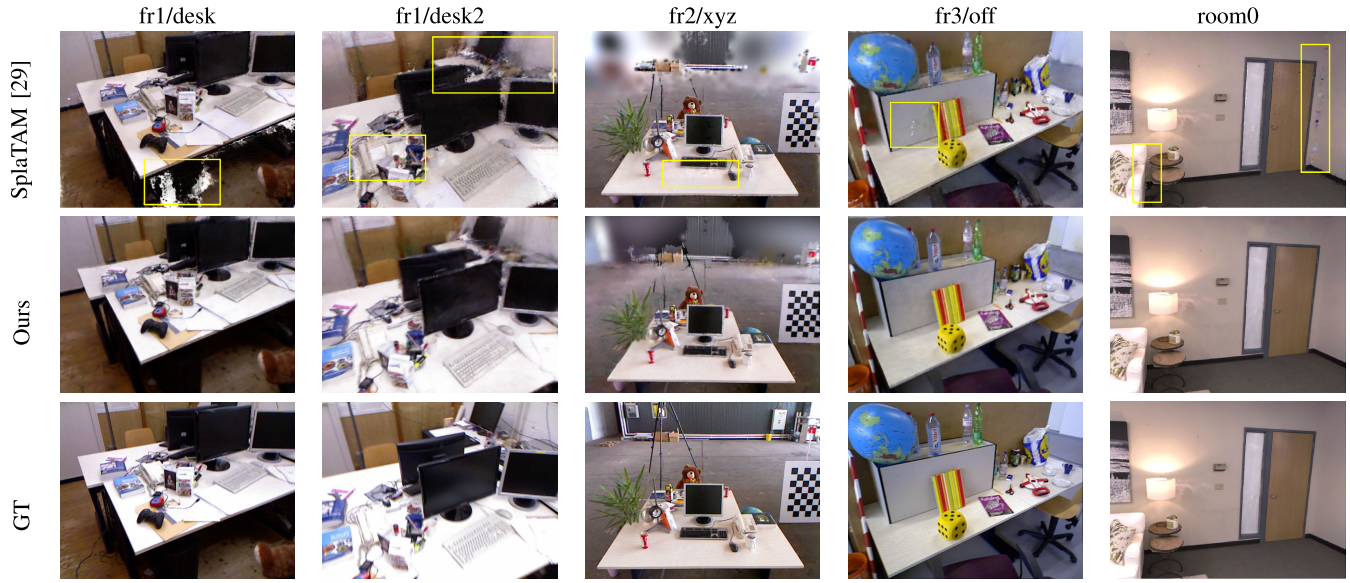

Fig. 4. **RGB-D rendering results.** Although SplaTAM [29] is the most advanced gaussian-based SLAM method, there are still some flaws in its rendering results. These include abnormal color points, blurry edges, and holes at the edges. Our method produces more realistic rendering results.

TABLE V
RENDERING PERFORMANCE ON R3LIVE-DATASET [41]. OUR METHOD HAS ACHIEVED COMPETITIVE RESULTS IN PSNR, SSIM, AND LPIPS COMPARED TO THE LATEST LIDAR-SPECIFIC GAUSSIAN SPLATTING METHODS [46], [47]

Method	Metric	campus_00	deg_00	deg_01	avg
3DGS[28]	PSNR[dB]↑	16.34	13.17	17.56	15.69
	SSIM↑	0.571	0.426	0.502	0.50
	LPIPS↓	0.314	0.375	0.341	0.34
Gaussian-LIC[47]	PSNR[dB]↑	25.27	22.47	23.49	23.74
	SSIM↑	0.805	0.794	0.811	0.84
	LPIPS↓	0.212	0.192	0.187	0.20
LVI-GS[46]	PSNR[dB]↑	24.47	22.60	23.84	23.63
	SSIM↑	0.776	0.818	0.850	0.81
	LPIPS↓	0.244	0.159	0.170	0.19
Ours	PSNR[dB]↑	25.68	21.55	24.19	23.80
	SSIM↑	0.714	0.737	0.805	0.75
	LPIPS↓	0.196	0.161	0.166	0.17

we use the fused LiDAR point clouds as the initial point cloud and the odometry as the initial poses. Figure 5 shows our rendering results, which clearly demonstrate that our method effectively fills the scene, producing more realistic rendering outcomes. Table V presents the quantitative results on the R3Live Dataset. Our approach maintains competitive performance even when compared to the latest LiDAR-specific methods such as LVI-GS [46] and Gaussian-LIC [47]. The subpar performance observed with 3DGS can be attributed to the accumulation of errors in the odometry poses, resulting in inconsistent observations. Conversely, our approach more accurately reconstructs the actual scene through the optimization of both poses and the scene.

E. Memory Comparison

SplaTAM [29], Gaussian Splatting SLAM [6] and GS-SLAM [34] add gaussian points according to the rendered depth and ground truth depth. Table IV indicates that this approach can lead to a rapid increase in memory usage, particularly with data exhibiting poor depth quality. The inconsistency in depth values tends to result in the continuous addition of points, making the parameters difficult to converge. Our rendering pipeline-oriented method only adds necessary points, avoiding the problem of memory overflow.

F. Ablation Study

1) View Synthesis: As shown in Table VI, we evaluate the impact of different strategies on a TUM-RGBD sequence *fr1/desk*. The results show that no matter in novel view or train view, isotropic regularization leads to a decrease in fidelity, as scale regularization conflicts with color loss, reducing the efficiency of color constraints. In contrast, isotropic regularization has a positive impact on depth optimization, as it constrains the number of affected tiles, thereby benefiting depth optimization.

Meanwhile, it can be observed that depth constraint has little impact on the fidelity of the training view, but significantly affects the fidelity of the novel view, as it eliminates depth ambiguity.

2) Uncertainty & Depth Prediction & Pre-Tracking: Table VIII examines the effects of depth prediction and the assignment of uncertainty on monocular scene optimization. We observe that scene optimization is substantially hindered by the absence of depth prediction, due to the difficulty of fulfilling the scene with only sparse points. Furthermore, by accounting for uncertainty, the problems caused by incorrect points in depth prediction are reduced.

Our photometric constraint frame-to-map optimization yields better accuracy than frame-to-frame pose optimization. If depth constraint is available, it is incorporated for potentially higher accuracy.

3) Depth-Based Scene Initialization: Scene initialization includes the initialization of the scale and rotation of the gaussian points. In the random initialization test, the random scale S is distributed according to $\mathcal{N}(0.05, 0.01)$. The random rotation is sampled uniformly in space according to the method in [48].

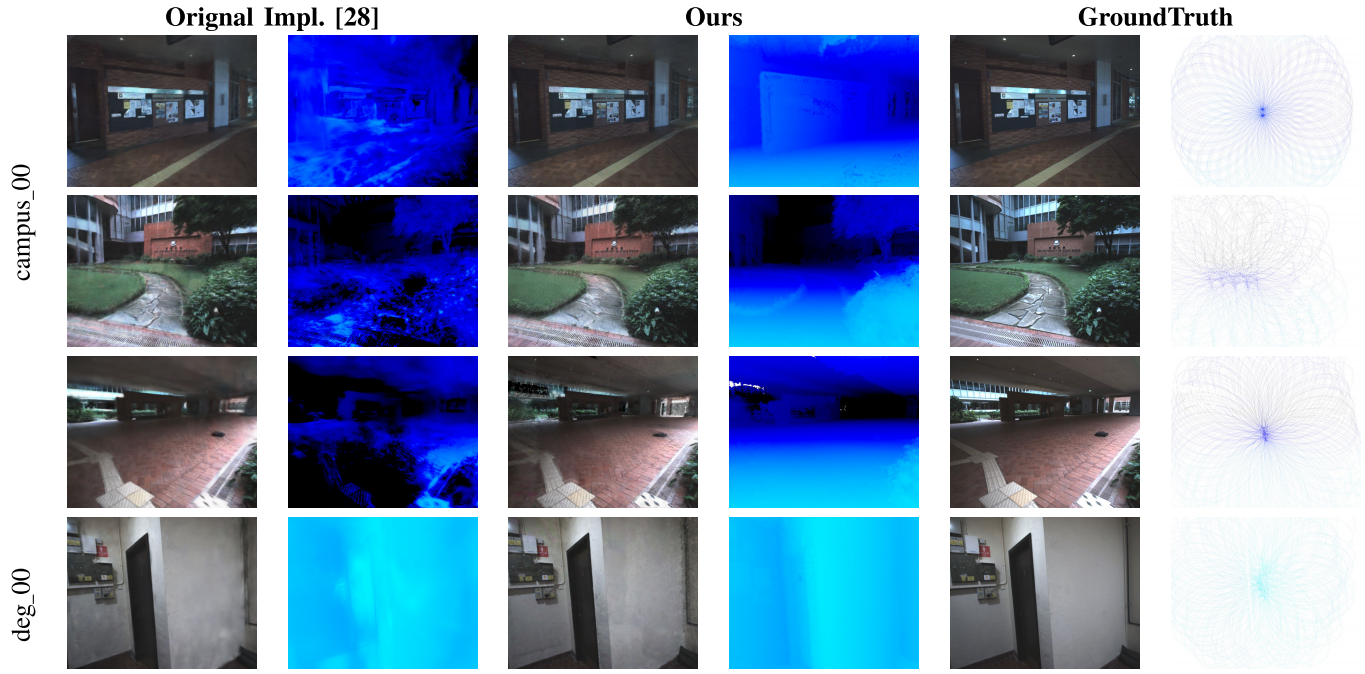

Fig. 5. **Rendering results on R3Live-Dataset [41].** Our method surpasses the original 3DGS [28], which relies on fused LiDAR point clouds for initial data. It demonstrates a significant improvement in accurately depicting the real scene depth.

TABLE VI
ABLATION STUDY ON VIEW SYNTHESIS. THE EVALUATION OF NOVEL VIEW SYNTHESIS IS CONDUCTED USING GROUND TRUTH POSES. THE RESULTS INDICATE THAT ISOTROPIC REGULARIZATION HAS A NEGATIVE IMPACT ON NOVEL VIEW SYNTHESIS, WHILE DEPTH CONSTRAINT HAS A POSITIVE IMPACT

	Training View				Novel View			
Metrics	PSNR[dB]↑	SSIM↑	LPIPS ↓	Depth L1(cm) ↓	PSNR[dB]↑	SSIM↑	LPIPS ↓	Depth L1(cm) ↓
Ours w/o depth constraint	25.43	0.832	1.51	10.3	19.95	0.677	0.231	13.5
Ours w/ isotropic regularization	24.22	0.821	1.56	**4.42**	20.42	0.689	0.211	**7.18**
Ours	**25.76**	**0.835**	**1.45**	4.51	**20.46**	**0.690**	**0.207**	8.07

TABLE VII
ABLATION STUDY ON SCENE INITIALIZATION

	Replica [36]				TUM-RGBD [35]				R3Live [41]			
Method	Room0		Office0		fr1/desk		fr1/desk2		cam. 00		deg. 02	
Full random	23k		25k		14k		21k		60k*		60k*	
Ours	13k	↑ 43.4%	12k	↑ 52.0%	9k	↑ 35.7%	13k	↑ 38.1%	19k	↑ 68.3%	22k	↑ 63.3%

TABLE VIII
ABLATION STUDY ON MONOCULAR UNCERTAINTY & DEPTH PREDICTION

Method	PSNR[dB]↑	SSIM↑	ATE[cm]↓
w/o uncertainty	22.73	0.793	2.69
w/o depth prediction	18.33	0.675	3.62
Ours	**24.79**	**0.824**	**2.58**

TABLE IX
ABLATION STUDY ON RGB-D LOCALIZATION ACCURACY (ATE, RMSE IN cm)

Method	**Avg.** ↓	fr1/desk	fr1/desk2	fr2/xyz	fr3/long
w/o pre-tracking	4.28	4.51	6.27	1.36	4.97
w/o mask	3.40	3.43	5.27	1.18	3.87
Ours	**1.21**	**1.45**	**2.17**	**0.32**	**0.90**

To evaluate the importance of scene initialization, the scene continues to be optimized even after all frames have been added, until the performance metrics stabilize. The efficiency gain from initialization is assessed by comparing the required number of iterations. Table VII displays the efficiency.

Generally, our initialization can converge faster and achieve better results. In particular, within the LiDAR dataset, an asterisk (*) signifies ongoing optimization, indicating that the metrics are in the process of improvement and have not yet attained the performance benchmark set by our approach.

V. CONCLUSION AND LIMITATIONS

We propose G²-Mapping, a general gaussian-based mapping method that supports multi-source data input, offering a comprehensive differentiable renderer and a dynamically expandable gaussian scene representation. Our comparative analysis indicates that the proposed method not only achieves state-of-the-art results for monocular and RGB-D data in positioning accuracy and rendering quality but also shows promising applicability to LVI data. We believe that G²-Mapping sets a new benchmark for future research in scene representation and rendering, and open up a new avenue for general gaussian splatting based SLAM.

Limitations: When monocular tracking fails, the system relies on the median of scene depth to maintain scale consistency. On the other hand, the depth estimation module uses linear mapping to calculate depth values, which can result in errors and subsequently lead to inaccurate Gaussian point initialization. Sparse-guided depth completion presents a promising solution to this problem, as it leverages the sparse points to provide clues for the depth completion network, potentially improving accuracy.

REFERENCES

[1] X. Yang, H. Li, H. Zhai, Y. Ming, Y. Liu, and G. Zhang, "Vox-Fusion: Dense tracking and mapping with voxel-based neural implicit representation," in *Proc. IEEE Int. Symp. Mixed Augmented Reality (ISMAR)*, Oct. 2022, pp. 499–507, doi: 10.1109/ISMAR55827.2022.00066.

[2] L. Liu, J. Gu, K. Z. Lin, T. Chua, and C. Theobalt, "Neural sparse voxel fields," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds. Curran Associates, Inc., 2020, pp. 15651–15663. [Online]. Available: https://proceedings.neurips.cc/paper files/paper/2020/file/b4b758962f17808746e9bb832a6fa4b8-Paper.pdf

[3] Y. Zhang, G. Chen, and S. Cui, "Efficient large-scale scene representation with a hybrid of high-resolution grid and plane features," *Pattern Recognit.*, vol. 158, Feb. 2025, Art. no. 111001.

[4] Z. Zhu et al., "NICE-SLAM: Neural implicit scalable encoding for SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 12776–12786. [Online]. Available: https://api.semanticscholar.org/CorpusID:245385791

[5] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint coordinate and sparse parametric encodings for neural real-time SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13293–13302.

[6] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian splatting SLAM," 2023, *arXiv:2312.06741*.

[7] R. A. Newcombe, S. Lovegrove, and A. J. Davison, "Dtam: Dense tracking and mapping in real-time," in *Proc. Int. Conf. Comput. Vis.*, 2011, pp. 2320–2327. [Online]. Available: https://api.semanticscholar.org/CorpusID:1336659

[8] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 3, pp. 611–625, Mar. 2018, doi: 10.1109/TPAMI.2017.2658577.

[9] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015, doi: 10.1109/TRO.2015.2463671.

[10] A. Dai, M. Nießner, M. Zollhöfer, S. Izadi, and C. Theobalt, "BundleFusion: Real-time globally consistent 3D reconstruction using on-the-fly surface re-integration," 2016, *arXiv:1604.01093*.

[11] V. Adrian Prisacariu et al., "A framework for the volumetric integration of depth images," 2014, *arXiv:1410.0925*.

[12] M. Keller, D. Lefloch, M. Lambers, S. Izadi, T. Weyrich, and A. Kolb, "Real-time 3D reconstruction in dynamic scenes using point-based fusion," in *Proc. Int. Conf. 3D Vis.*, Jun. 2013, pp. 1–8, doi: 10.1109/3DV.2013.9.

[13] T. Schöps, T. Sattler, and M. Pollefeys, "BAD SLAM: Bundle adjusted direct RGB-D SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 134–144, doi: 10.1109/CVPR.2019.00022.

[14] J. Jiao et al., "Real-time metric-semantic mapping for autonomous navigation in outdoor environments," *IEEE Trans. Autom. Sci. Eng.*, pp. 1–12, Aug. 2024. [Online]. Available: http://dx.doi.org/10.1109/TASE.2024.3429280

[15] S. Zhi, M. Bloesch, S. Leutenegger, and A. J. Davison, "SceneCode: Monocular dense semantic reconstruction using learned encoded scene representations," 2019, *arXiv:1903.06482*.

[16] Z. Teed and J. Deng, "DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds. Curran Associates, Inc., 2021, pp. 16558–16569. [Online]. Available: https://proceedings.neurips.cc/paper files/paper/2021/file/89fcd07f20b6785b92134bd6c1d0fa42-Paper.pdf

[17] R. A. Newcombe et al., "KinectFusion: Real-time dense surface mapping and tracking," in *Proc. 10th IEEE Int. Symp. Mixed Augmented Reality*, Oct. 2011, pp. 127–136, doi: 10.1109/ISMAR.2011.6092378.

[18] J. McCormac, A. Handa, A. Davison, and S. Leutenegger, "SemanticFusion: Dense 3D semantic mapping with convolutional neural networks," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2017, pp. 4628–4635, doi: 10.1109/ICRA.2017.7989538.

[19] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proc. Eur. Conf. Comput. Vis.*, Aug. 2020, pp. 405–421, doi: 10.1007/978-3-030-58452-8_24.

[20] M. Niemeyer, L. Mescheder, M. Oechsle, and A. Geiger, "Differentiable volumetric rendering: Learning implicit 3D representations without 3D supervision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 3501–3512, doi: 10.1109/CVPR42600.2020.00356.

[21] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, "Plenoxels: Radiance fields without neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 5491–5500, doi: 10.1109/CVPR52688.2022.00542.

[22] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 1–15, Jul. 2022, doi: 10.1145/3528223.3530127.

[23] G. Kopanas, J. Philip, T. Leimkühler, and G. Drettakis, "Point-based neural rendering with per-view optimization," *Comput. Graph. Forum*, vol. 40, no. 4, pp. 29–43, Jul. 2021, doi: 10.1111/CGF.14339.

[24] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Mip-NeRF 360: Unbounded anti-aliased neural radiance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 5460–5469.

[25] M. Tancik et al., "Block-NeRF: Scalable large scene neural view synthesis," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 8238–8248, doi: 10.1109/CVPR52688.2022.00807.

[26] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "IMAP: Implicit mapping and positioning in real-time," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 6209–6218.

[27] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald, "Point-SLAM: Dense neural point cloud-based SLAM," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 18387–18398.

[28] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–14, Aug. 2023.

[29] N. Keetha et al., "SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM," 2023, *arXiv:2312.02126*.

[30] S. Hong, J. He, X. Zheng, C. Zheng, and S. Shen, "LIV-GaussMap: LiDAR-inertial-visual fusion for real-time 3D radiance field map rendering," 2024, *arXiv:2401.14857*.

[31] Y. Hu, T.-M. Li, L. Anderson, J. Ragan-Kelley, and F. Durand, "Taichi: A language for high-performance computation on spatially sparse data structures," *ACM Trans. Graph.*, vol. 38, no. 6, p. 201, 2019.

[32] M. Zwicker, H. Pfister, J. van Baar, and M. Gross, "EWA splatting," *IEEE Trans. Vis. Comput. Graphics*, vol. 8, no. 3, pp. 223–238, Jul. 2002, doi: 10.1109/TVCG.2002.1021576.

[33] C.-H. Lin, W.-C. Ma, A. Torralba, and S. Lucey, "BARF: Bundle-adjusting neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5741–5751.

[34] C. Yan et al., "GS-SLAM: Dense visual SLAM with 3D Gaussian splatting," 2023, *arXiv:2311.11700*.

[35] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 573–580.

[36] J. Straub et al., "The replica dataset: A digital replica of indoor spaces," 2019, *arXiv:1906.05797*.

[37] T. J. Whelan, M. Kaess, M. Fallon, H. Johannsson, J. J. Leonard, and J. McDonald, "Kintinuous: Spatially extended KinectFusion," in *Proc. Nat. Conf. Artif. Intell.*, Jul. 2012.

[38] T. Whelan, R. F. Salas-Moreno, B. Glocker, A. J. Davison, and S. Leutenegger, "ElasticFusion: Real-time dense SLAM and light source estimation," *Int. J. Robot. Res.*, vol. 35, no. 14, pp. 1697–1716, Dec. 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:21124365

[39] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual–inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.

[40] R. Wang, M. Schworer, and D. Cremers, "Stereo DSO: Large-scale direct sparse visual odometry with stereo cameras," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2017, pp. 3903–3911.

[41] J. Lin and F. Zhang, "R^{3}LIVE: A robust, real-time, RGB-colored, LiDAR-inertial-visual tightly-coupled state estimation and mapping package," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 10672–10678.

[42] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 586–595.
[43] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
[44] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient dense SLAM system based on hybrid representation of signed distance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 17408–17419.
[45] Z. Zhu et al., "NICER-SLAM: Neural implicit scene encoding for RGB SLAM," 2023, *arXiv:2302.03594*.
[46] H. Zhao, W. Guan, and P. Lu, "LVI-GS: Tightly-coupled LiDAR-visual-inertial SLAM using 3D Gaussian splatting," 2024, *arXiv:2411.02703*.
[47] X. Lang et al., "Gaussian-LIC: Real-time photo-realistic SLAM with Gaussian splatting and LiDAR-inertial-camera fusion," 2024, *arXiv:2404.06926*.
[48] K. Shoemake, "Animating rotation with quaternion curves," in *Proc. 12th Annu. Conf. Comput. Graph. Interact. Techn. (SIGGRAPH)*, 1985, pp. 245–254.

Lin Chen received the B.E. degree from the College of Aeronautics, Northwestern Polytechnical University (NPU), Xi'an, China, in 2019, where he is currently pursuing the Ph.D. degree. His research interests are computer vision and robotics, including 3-D reconstruction, Gaussian splatting simultaneous localization and mapping (SLAM), and related fields.

Boni Hu received the bachelor's degree from Northwest A&F University, Xianyang, China, in 2018, and the master's degree from Northwestern Polytechnical University, Xi'an, China, in 2021, where she is currently pursuing the Ph.D. degree. Her research interests include visual geolocalization and image retrieval.

Jvboxi Wang received the bachelor's degree from the College of Aeronautics, Northwestern Polytechnical University (NPU), Xi'an, China, in 2024, where he is currently pursuing the master's degree. His research interests include robotics, Gaussian splatting, and related fields.

Shuhui Bu (Member, IEEE) received the M.Sc. and Ph.D. degrees in computer science from the College of Systems and Information Engineering, University of Tsukuba, Tsukuba, Japan, in 2006 and 2009, respectively. He is currently a Professor with Northwestern Polytechnical University, Xi'an, China. He has authored or co-authored approximately 80 papers in major international journals and conferences. His research interests include computer vision and robotics, including simultaneous localization and mapping (SLAM), 3-D shape analysis, image processing, pattern recognition, 3-D reconstruction, and related fields.

Guangming Wang (Graduate Student Member, IEEE) received the B.S. degree from the Department of Automation, Central South University, Changsha, China, in 2018, and the Ph.D. degree in control science and engineering from Shanghai Jiao Tong University, Shanghai, China, in 2023. He visited Department of Computer Science of ETH Zurich for one year. He is currently a Research Associate with the Department of Engineering, University of Cambridge, U.K. His current research interests include SLAM and computer vision. He is an Associate Editor (AE) of IEEE ROBOTICS AND AUTOMATION LETTERS. He also served as an Associate Editor (AE) for conferences, like ICRA2024, 2025, and IROS2024, 2025.

Pengcheng Han received the Ph.D. degree from Northwestern Polytechnical University, Xi'an, China, in 2023. Currently, he is engaged as a Post-Doctoral Researcher with Northwestern Polytechnical University. His research interests include aircraft perception and decision-making, machine learning, and computer vision.

Jian Chen is currently pursuing the Ph.D. degree with the School of Clinical Medicine, University of Cambridge. Specifically, he is interested in 3-D reconstruction, multi-modal, and weakly-supervised methods applied in diagnosis. His primary focus is on medical imaging.